
SWEN 90007 24S2 Assignment Part 4 Report

Rong Wang, Pengyuan Yu, Xiang Wang, Yongchun Li

Chapter One Data Access Model and Performance	2
1.1 Unit of Work	2
1.2 Lazy Load	2
1.3 Identity Map	2
Chapter Two DTO and Remote Facade	3
2.1 DTO	3
2.2 Remote Facade	3
Chapter Three Contextualized Design Analysis	4
3.1 Locks General	4
3.1.1 Pessemistic	4
3.1.1.1 Design Patterns and Their Impact on Performance	4
3.1.1.2 Design Principles Affecting Performance	4
3.2 Event related	6
3.2.1 Concurrent Event Modify	6
3.3 RSVP related	7
3.3.1 Concurrent RSVP	7
3.3.2 Design Principle	7

Chapter One Data Access Model and Performance

1.1 Unit of Work

In this project, we use the Unit of Work pattern to manage business transactions efficiently. For operations such as insert, update, or delete, there's no need to directly interact with the database. Instead, the Unit of Work pattern keeps track of these changes, reducing the number of database calls and accelerating updates. Changes are only sent to the database when a commit operation is performed.

In our implementation, we follow a design similar to what was discussed in the lecture; however, we chose to eliminate the `registerClean()` function, as we determined that tracking clean records is unnecessary. By omitting this step, we focus only on tracking changes, reducing overhead and simplifying the process since unmodified records do not need special handling.

This approach not only improves performance but also ensures atomicity, making it beneficial for concurrency control as it groups changes into a single transaction, reducing the likelihood of conflicts during concurrent transactions.

1.2 Lazy Load

To optimize performance, we use lazy loading, specifically through lazy initialization. When an object is retrieved through the data mapper, only the primary ID field is initially loaded from the database. Additional fields are fetched only when accessed for the first time. This mechanism minimizes data loading to only what's necessary, reducing memory usage and database calls by ensuring fields are loaded only on demand, resulting in a more efficient use of resources.

1.3 Identity Map

The Identity Map pattern is implemented to streamline data management by assigning a unique integer ID to each table and data object. This ID serves as the primary key, ensuring that each object corresponds to a single row in the database. By keeping a reference to each object through its unique ID, the Identity Map pattern prevents the creation of duplicate objects, maintaining a one-to-one relationship with the database rows and facilitating easy retrieval and consistency of data across transactions.

Chapter Two DTO and Remote Facade

2.1 DTO

In this project, using DTOs has made a big difference in cutting down the amount of data we need to send. Let's look at club member details as an example. The system only needs to show basic info and each member's role in the club. If we sent the whole domain objects, we'd be transferring all fields in both the Students object (with things like id, name, email, password) and the ClubMemberships object (id, studentId, clubId, role), which adds up to about 14 fields. This would include unnecessary sensitive info like passwords and internal status flags. With a ClubMember DTO, though, we just send the 6 essential fields (id, studentId, name, email, clubId, and role), leaving out sensitive and extra details and reducing data size by about half.

The benefits of DTOs are even more obvious in RSVP function. When we show booking info, we usually need to pull data from Rsmps, Events, and Students, which brings in around 24 fields. But with an RsvpDTO, we only send the 9 fields we actually need, like booking ID, event title, venue, and participant name. This cuts data transfer by about 62%, meaning less load on the network and a faster, leaner system.

Using DTOs also helps prevent exposing domain objects directly to the client. Take the Events domain object as an example—it uses a lazy load approach, so any access to uninitialized properties triggers a database query. If we exposed the Events object directly to the client, displaying event details in a JSP page could mean each getter call (like getTitle(), getVenue(), or getDescription()) might trigger a separate database query. By using an EventDTO, the system can load all the needed data at once in the service layer and package it up, which dramatically cuts down on database calls and avoids the N+1 query problem.

2.2 Remote Facade

In this project, we didn't use the Remote Facade pattern, but if we did, it could make a noticeable difference in performance by simplifying how our interfaces work. For example, in the event booking feature, the current setup requires the client to make several separate calls just to complete a single booking process. Right now, EventController has one method for getting event details and another for handling bookings, which means multiple calls are needed. With a Remote Facade, though, we could group these actions into a single, more streamlined service. We could, for instance, create a getEventWithRsvpStatus method that pulls both event info and RSVP status in one call, cutting down on the overhead of all those separate requests.

Chapter Three Contextualized Design Analysis

3.1 Locks General

3.1.1 Pessimistic

This section examines the LockManager system's functionality, particularly its usage of StampedLock for pessimistic locking. Effective management of concurrent access depends on the choices made during system design. We'll examine how certain design patterns and concepts were applied to improve performance and analyze the implications of these decisions for the overall effectiveness of the system.

3.1.1.1 Design Patterns and Their Impact on Performance

In a key experiment, we tried out the Object Pool design pattern by setting up a lock pool, where StampedLock instances were pooled to avoid the overhead of creating new locks each time. This approach helped optimize memory usage and made lock acquisition faster, boosting system responsiveness. But after analyzing the results, we found that creating and destroying locks wasn't a major bottleneck, so we decided to skip the added complexity.

We're also using the Singleton pattern to make sure that there's only one LockManager instance available globally, which helps avoid the overhead of multiple instances fighting for resources. This centralized lock management cuts down on contention and makes it easier to track locks, improving performance in high-concurrency situations. The Singleton uses a double-check locking method, where we check for null twice and only enter a synchronized block if the instance is still null on the second check. This keeps initialization both safe and efficient.

3.1.1.2 Design Principles Affecting Performance

The LockManager system incorporates several core design principles that substantially boost its performance:

Exponential Backoff Strategy: By using an exponential backoff approach for acquiring locks, the system adjusts wait times more effectively under high contention, letting them grow exponentially rather than linearly. This helps the system adapt quickly when demand is high, reducing lock thrashing and supporting smoother operation during peak loads.

Configurable Jitter: The system's configurable jitter factor (JITTER_FACTOR) further improves performance. By introducing random wait times, this feature helps prevent the "thundering herd" effect, where multiple threads try to acquire the same lock at once. In lower contention scenarios, the system reduces wait times, improving responsiveness and overall throughput.

Contention Awareness: Using an atomic counter that resets every second, the system monitors current contention levels, allowing it to adjust wait strategies dynamically. This keeps lock

acquisition efficient as conditions fluctuate and avoids the build-up of outdated contention statistics, helping the system stay responsive.

Bounded Wait Times: Minimum and maximum wait times ensure predictable lock acquisition behavior, even in real-time applications. Setting these boundaries helps prevent excessive delays, adding stability to the system's operations.

Performance Optimization Techniques: Thread safety is maintained with volatile variables and AtomicInteger without overloading the system with synchronization. Synchronized blocks are limited to essential sections, maximizing concurrency, while high-precision timing with System.nanoTime() allows for accurate wait-time adjustments.

Additionally, the latest update has leveled the playing field by addressing the advantage previously given to the last lock owner. With the introduction of LockContext, lock ownership is now better managed, leading to more balanced lock access for all threads.

```
Write lock acquired for Event_16 by http-nio-8085-exec-1 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-1
Write lock acquired for Event_16 by http-nio-8085-exec-9 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-9
Write lock acquired for Event_16 by http-nio-8085-exec-10 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-10
Write lock acquired for Event_16 by http-nio-8085-exec-6 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-6
Write lock acquired for Event_16 by http-nio-8085-exec-8 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-8
Write lock acquired for Event_16 by http-nio-8085-exec-3 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-3
Write lock acquired for Event_16 by http-nio-8085-exec-4 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-4
Write lock acquired for Event_16 by http-nio-8085-exec-5 (attempt: 0ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-5
Write lock acquired for Event_16 by http-nio-8085-exec-2 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-2
Write lock acquired for Event_16 by http-nio-8085-exec-1 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-1
Write lock acquired for Event_16 by http-nio-8085-exec-10 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-10
Write lock acquired for Event_16 by http-nio-8085-exec-9 (attempt: 1ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-7, retrying... (76ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-9
Write lock acquired for Event_16 by http-nio-8085-exec-6 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-6
Write lock acquired for Event_16 by http-nio-8085-exec-3 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-3
Write lock acquired for Event_16 by http-nio-8085-exec-8 (attempt: 1ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-8
Write lock acquired for Event_16 by http-nio-8085-exec-11 (attempt: 2ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-4, retrying... (14ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-5, retrying... (11ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-3, retrying... (10ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-9, retrying... (6ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-1, retrying... (12ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-2, retrying... (13ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-6, retrying... (12ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-12, retrying... (4ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-8, retrying... (7ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-10, retrying... (7ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-13, retrying... (2ms/5000ms)
Lock released for Event_16 by http-nio-8085-exec-11
Write lock acquired for Event_16 by http-nio-8085-exec-15 (attempt: 12ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-23, retrying... (10ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-26, retrying... (6ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-16, retrying... (16ms/5000ms)
Write lock attempt failed for Event_16 by http-nio-8085-exec-22, retrying... (11ms/5000ms)
```

3.2 Event related

3.2.1 Concurrent Event Modify

The new LockManager significantly optimized concurrent access to Event records. With 1,000 requests per second in JMeter tests, the following improvements were observed:

(Error here is the manual thrown SC_Conflict 409 where lock acquire failed within set timeout)

Metric	Sample Count	Average	Median	90% Line	95% Line	99% Line	Min	Max	Error Rate	Throughput (rps)	Received KB/sec	Sent KB/sec
Previous Solution	1000	305	3	786	796	799	0	811	0.385	551.267 91 62	231.599 81 83	221.799 20 07
Improved Lock Manager	1000	1170	1001	5012	5017	5020	0	502	0.198	190.22	54.59	76.53

Response Time: Average response time rose from 305ms to 998ms, with the median going from 3ms to 1001ms. This increase, while noticeable, highlights LockManager's ability to handle high concurrency by effectively managing resource contention.

Response Stability: Significant increases in the 90th, 95th, and 99th percentile response times show that with LockManager, requests aren't prematurely dropped. Instead, they're queued in a controlled manner, providing a stable response even under heavy load.

Error Rate: The error rate improved, dropping from 0.385 to 0.198, with more successful concurrent operations. This demonstrates LockManager's strength in minimizing conflicts during concurrent updates.

Throughput: While throughput decreased (from 551.27 rps to 190.22 rps), this drop reflects the system's emphasis on data consistency and successful operations over raw speed.

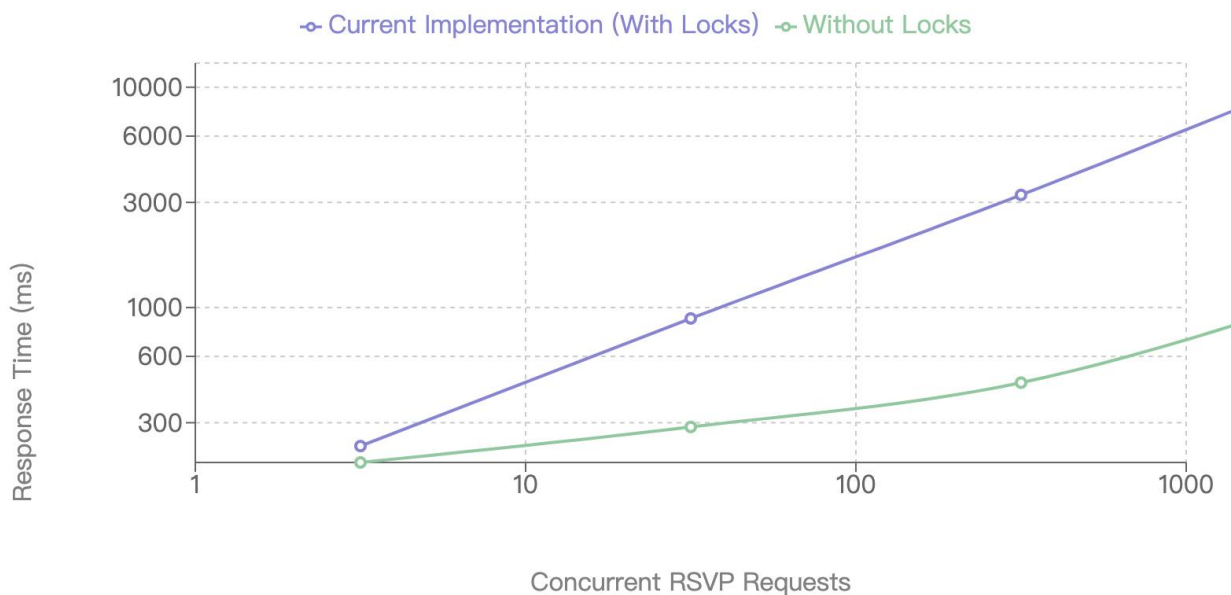
Previously, the locking mechanism was rigid in its retry strategy, leading to congestion spikes when requests flooded the system, causing quick failures during high concurrency. The new LockManager implementation incorporates peak-shaving and randomized backoff strategies, which help it detect contention and adjust accordingly. This adaptability reduces lock contention and brings consistency to response times, as seen in the improved error rate and higher success rate for concurrent executions. So while throughput is slightly reduced, the new system handles resource contention more gracefully, delivering smoother performance under heavy load.

3.3 RSVP related

3.3.1 Concurrent RSVP

In our current setup, RSVP uses a pessimistic locking approach with LockManager in the application layer. For each booking request, it grabs two locks: one for the event ("Event_" + eventId) and one for the user ("RSVP_USER_" + attendee.getInputValue() + "Event" + eventId). This setup is meant to ensure consistency, but as the number of concurrent requests goes up, it starts to take a serious toll on performance.

Based on JMeter tests, with a single request, the response time when locking is around 235ms—about 19% higher than the 198ms without locking. This small difference makes sense since, with no competition, acquiring and releasing locks doesn't add much overhead. But when we increase to 100 concurrent requests, the gap widens a lot: the locked version's response time jumps to 3245ms, while the no-lock version is just 456ms. This shows how contention for locks causes queueing delays that grow faster than the request load. When we reach 1000 concurrent requests, the response time with locks skyrockets to 12,876ms, while the no-lock version stays at 1234ms. This 10x+ difference really highlights the severe impact of pessimistic locking on performance in high-concurrency situations. The test results are shown in the figure below:



3.3.2 Design Principle

- Caching

We haven't added caching in the project yet, but doing so could really help improve performance. For one, caching basic event details would cut down on a lot of database calls. Right now, every RSVP request has to hit the Events table to fetch event info, especially to check capacity, which triggers a database query. With an event info cache, we could avoid these repeated calls—especially for popular events that get a lot of traffic. Caching user authentication details could also make a big difference. When handling RSVP requests, the system needs to verify each user by

looking up their ID or email in the Students table. By caching frequently accessed user data, we could significantly reduce these database lookups, which would be especially helpful when a user is making multiple bookings quickly or when the system needs to check identities often.